

Table of Contents

Sr.	Description	Page
1	Abstract	1
2	Introduction	2
3	Literature Review	4
4	Methodology	6
5	Results and Discussion	15
6	References	26

Prepared by	: Zarni Paing Sett
-------------	--------------------

Abstract

The objective of this project is to perform customer segmentation using K-means clustering on an e-commerce dataset based on customer lifestyle and demographics. The dataset consists of customer characteristics such as age, family size, annual income, and spending scores. Data preprocessing included filling in missing values, outlier detection, and normalising the features. Two main clustering were performed in this prototype: one using annual income and spending scores, and another using age and family size. Optimal number of clusters for clustering was determined using the Elbow Method and Silhouette Score. The clustering produced seven groups of customers for lifestyle segmentation and four groups for demographic segmentation. The results were presented visually with defined centroids and segment names to help with interpretation of these results, which offer a good perspective on how customers activity patterns and behaviours are interrelated and can be important in formulating marketing strategies and customer relationship management.

Introduction

The fast growth of e-commerce has led to an enormous of data about customers to the benefit of businesses, which have a far richer information ecosystem to operate and target their audiences effectively now, more than ever. One of the most fundamental ways of leveraging this data is through customer segmentation which is the grouping of customers into two or more homogeneous groups in order to make marketing and engagement strategies. In this project, I aim to focus on dividing an e-commerce customer base into unique segments in order to provide useful findings about their behaviours and preferences. It is through this segmentation that businesses and companies can satisfy the needs of each customer group leading to enhanced customer satisfaction, loyal, and eventually, an increase in sales.

Given that customers vary greatly from one another in their behaviours, preferences, and demographics, customer segmentation becomes essential as understanding the why and how of adjusting marketing strategies, products, and services for different customer groups can lead to significant improvements. Digitalisation has made it easy for one to collect a lot of the customer data, but to understand, interpret, and derive useful value from this data still remains complex. Traditional means of segmentations are quite useful but are very often not detailed enough to capture high complexities of customer behaviours within vast and different customer bases. Advanced machine learning techniques such as K-means clustering can present an extensive solution to businesses, enabling them to divide and classify their customers in segments based on numerous factors such as spending behaviour, income, age, and family size. This segmentation approach helps the marketers develop strategies specific to each segment which in turn helps in the better management of the available resources and increases the success of the marketing campaigns.

I have decided to use K-means clustering algorithm for this project due to the widespread use of K-means and its ability to handle large amounts of data very well. K-means is also fast which is appropriate for high-dimensional data and the output is given in a cluster form which can be used for practical business insights. This project aims to simplify and improve the understanding and representation of such complex customer information by concentrating on the patterns of spending behaviour and demographic characteristics of customers.

This project applies techniques for customer segmentation to an e-commerce dataset and the analysis is split into two main clustering models: lifestyle clustering and demographic clustering. To begin with, the dataset is first cleansed and preprocessed in order to maintain quality, which involves handling of missing values, elimination of duplicates, outlier detection, and normalisation of numeric attributes using Min-Max Scaling for optimal clustering performance. Two sets of features are used to obtain different customer perspectives and insights. For this, Annual Income and Spending Score are selected for lifestyle segmentation, while Age and Family Size are selected for demographic segmentation. These features are central in understanding how customers tend to spend in relation to their income, and how spending is influenced by their age and family size. These segmentations provide descriptive statistics and visualisations for each customer segment within the customer base. It is essential to determine the optimal number of clusters for meaningful segmentation. Each of the segmentation types is evaluated for the appropriate cluster counts by using the Elbow Method and Silhouette Score analysis. The Elbow Method examines the rate of change in the within-cluster sum of squares, while Silhouette Score measures the coherence of the clusters. Lifestyle Clustering segments customers into seven spending-based clusters (e.g., “High Spending, Low Income” and “Affluent High Spenders”), whereas Demographic Clustering groups customers by age and family structure, placing them into four cluster types (e.g., “Young, with Small Family”) to help understand the spending patterns and the demographics better.

The clustered data is visualised using scatter plots, bar charts, and box plots in order to demonstrate the distribution of customer segments and centroids, which in turn, improves the interpretability of clusters and their possible influence on strategic business decisions. With the help of such segmentation, businesses can develop detailed customer profiles. This means that it would be easier for businesses to promote advertisements according to certain needs, design loyalty programs for customers belonging to high-spending clusters, and carry out targeted sales promotions, thereby aligning resources with customer priorities to maximise marketing efficiency and strengthen customer relationships. As a result, this project provides a robust framework for customer segmentation which can be used across various e-commerce services and activities. The results provide a thorough analysis of clustering and detailed feature, which is essential for improving customer interactions and customer relationship management in the e-business space.

Literature Review

Zhao (2024) states that customer segmentation plays a major role in improving marketing techniques by classifying customers into different groups on the basis of parameters such as age and buying behaviour. In the past few decades, owing to the rapid advancements in the domain of artificial intelligence, particularly machine learning, K-means clustering has emerged as a simple but effective technique for the task of customer segmentation, which is an essential part of marketing and classifies customers with respect to the distance of points from each other. Nevertheless, K-means contains some limitations particularly the issue of sensitivity to initial cluster centers which may lead to clusters of different quality being produced. A different choice of parameter k could also lead to a different clustering result.

The use of K-means and its variations has become ubiquitous in the business field in terms of identifying the appropriate marketing strategies and efficient allocation of funds. Although effectively used, Zhao (2024) emphasises that data quality problems and the complexity of customer data problems persist, and further states that based on the trends of artificial intelligence development, especially after the introduction of deep learning, it is likely that within not-so-distant future customer segmentation could become more adaptive and capable of capturing the nuances of customer behaviour more effectively as the newly developed specialised clustering algorithms continue to emerge.

K-means is an unsupervised algorithm widely valued for its speed and adaptability to large datasets where customers can be grouped according to their characteristics and the way they spend money. Madhu et al. (2021) demonstrated this by clustering customers based on their spending on products (such as Grocery, Milk, Fresh, etc.), identifying groups such as high spenders and low spenders. In my project, a similar approach is used to segment customers by family size and age and give descriptive labels in order to make the result more interpretable. Effective K-means clustering requires precise, clean, and relevant data. Madhu et al. (2021) excluded irrelevant data and instead focused on qualitative variables that are directly linked to expenditure in order to help enhance segmentation accuracy. Similarly in this project, I applied a clean dataset and used the Elbow Method and Silhouette Score to determine the optimal number of clusters to use. This approach helps in creating a well-balanced model, minimising overfitting, and supporting meaningful segmentation.

In the study by Tabianan et al. (2022), the authors explore behavioural factors with an emphasis on analysing purchasing habits to refine marketing strategies. Their research proposes a hybrid approach that combines K-means with an improved algorithm called SAPK, which improves clustering performance by refining and adjusting cluster centers for more accurate segmentation of e-commerce customers. The purpose of their study is to evaluate customer purchasing patterns and their segmentation within the context of e-commerce for the benefit of providing insights for researchers and other stakeholders in e-commerce on structuring purchasing patterns. It aims to divide e-commerce behaviour data from multiple categories for the purpose of highlighting inequalities between high-profit and low-profit segments. The results reveal that although the SAPK + K-Means algorithm demonstrates a low error rate compared to the AP + K-Means method and traditional K-Means methods, the longer processing time associated with the complexity of the dataset is justified by its increased accuracy (Tabianan, et al., 2022).

A study of 520 online customers in Turkey by Akar (2021) highlights how pandemic-related concerns influence online shopping intentions by extending the Theory of Planned Behaviour (TPB) to demonstrate their impact on perceived behavioural control, subjective norms, and behavioural attitudes, all of which affected purchase intentions. The study employed path analysis and segmented the customers into three concern-based groups using the k-means algorithm, which showed that during times of crisis, psychological and emotional aspects are quite significant in guiding customer behaviours. Such details are particularly helpful to e-commerce stakeholders to tailor marketing strategies to changing customer behaviours and propose further areas of study in online shopping (Akar, 2021).

This paper by Alves Gomes and Meisen (2023) reviews 105 publications from 2000 to 2022 on customer behaviour segmentation, outlining a four-phase process: data collection, customer representation, segmentation, and targeting. K-means clustering is the most preferred method because of its simplicity and scalability, yet the study highlights the lack of a universally applicable segmentation method for all e-commerce contexts and emphasises the need for better evaluation metrics of segmentation performance across various applications (Alves Gomes & Meisen, 2023).

In summary, while K-means clustering is effective for customer segmentation, it faces limitations such as sensitivity to initial clusters and challenges with complex data, prompting the need to explore advanced hybrid approaches and deep learning techniques.

Methodology

This project on customer segmentation consisted of individual data preprocessing and analysis, as well as clustering, in order to group customers according to their spending habits and demographic characteristics. The methods and techniques used to complete the project were selected very carefully in order to make clustering not only effective, but also clear in its meaning.

First, I set up the necessary libraries for data analysis and clustering (Figure 1). The purpose was to perform data preprocessing and visualisation, and to implement K-means clustering algorithm. MinMaxScaler was used to normalise the data before clustering so that all scaled features would contribute equally to the distance calculations used in clustering algorithms.

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from pathlib import Path
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import MinMaxScaler
```

Figure 1

The dataset, in CSV format, was imported using Pandas, and basic checks were performed to understand its dimensions and column names, followed by preliminary visualisations and summaries to gain an initial understanding of the data (Figure 2).

```
url = 'C:/Users/Lenovo/Desktop/BUC/1st Sem/CET 313 - Artificial Intelligence/ASSIGNMENT/Customer.csv'

# List files in the directory
print([file.name for file in Path(url).parent.iterdir() if file.is_file()])

# Read the CSV file
customer_data = pd.read_csv(url)

# Print DataFrame dimensions
print(f'Dataframe dimensions: {customer_data.shape}')

✓ 0.0s

['Customer.csv', 'customersegmentation.ipynb', 'notes.txt']
Dataframe dimensions: (2000, 8)

# Display the first 15 rows of the data
display(customer_data.head(15))

✓ 0.0s
```

Figure 2

As shown in Figure 3, the 'CustomerID' column of the customer_data DataFrame is checked for null values, the existing nulls are counted and printed, the dimension and content of the DataFrame are displayed, duplicates are identified, and finally, a summary DataFrame containing information regarding each column types, the number of nulls in each column and the nulls' percentage is created.

```
# Check for null values in 'CustomerID' and display relevant info
nan_count_customer_id = customer_data['CustomerID'].isnull().sum()

print(f'NaN values in CustomerID: {nan_count_customer_id}')

# Display the total number of entries and the dimensions of the DataFrame
print(f'Dataframe dimensions: {customer_data.shape}')

✓ 0.0s
NaN values in CustomerID: 0
Dataframe dimensions: (2000, 8)

# Check for duplicates in the DataFrame
duplicate_count = customer_data.duplicated().sum()
print(f'Duplicates: {duplicate_count}')

✓ 0.0s
Duplicates: 0

# Create a DataFrame with column types, null values count, and null values percentage
data_info = pd.DataFrame({
    'column type': customer_data.dtypes,
    'null values (nb)': customer_data.isnull().sum(),
    'null values (%)': customer_data.isnull().mean() * 100
})
```

Figure 3

Null values in the 'Profession' column were identified (Figure 4) and they were filled using the mode (most frequent value) which is a straightforward approach to deal with any categorical missing data (Figure 5).

```
# Display the info DataFrame
display(data_info)
```

✓ 0.0s

	column type	null values (nb)	null values (%)
CustomerID	int64	0	0.00
Gender	object	0	0.00
Age	int64	0	0.00
Annual Income (\$)	int64	0	0.00
Spending Score (1-100)	int64	0	0.00
Profession	object	35	1.75
Work Experience	int64	0	0.00
Family Size	int64	0	0.00

Figure 4

```

# Create a mask for rows where 'Profession' was originally NaN
mask = customer_data['Profession'].isna()

# Store the original values before filling
original_values = customer_data.loc[mask, 'Profession'].copy()

# Fill missing values in the 'Profession' column with the mode
customer_data['Profession'] = customer_data['Profession'].fillna(customer_data['Profession'].mode()[0])

# Create a DataFrame to show which rows were filled
filled_rows = customer_data[mask].copy()
filled_rows['Original_Profession'] = original_values
filled_rows['Filled_Profession'] = customer_data.loc[mask, 'Profession']

# Display the rows where values were filled
display(filled_rows)

# Filter the customer data to retrieve rows where the 'Profession' column is still null
remaining_nulls = customer_data[customer_data['Profession'].isnull()]

# Display the remaining rows with null values
display(remaining_nulls)

```

✓ 0.0s

Figure 5

The code showcased in Figure 6 determines the frequency of each profession in the 'Profession' column of the customer_data DataFrame, uses Seaborn to create a horizontal bar plot visualising the distribution of the data, incorporating the titles and labels for better understanding (Figure 7). The creation of visualisations such as histograms and bar plots aids in looking through the distribution patterns of variables such as Gender, Profession, Annual Income, Spending Score, Age, and Family Size. This provides insights into customer demographics and their spending behaviours.

```

# Count occurrences of each profession
profession_distribution = customer_data['Profession'].value_counts()

# Create a horizontal bar plot with a color palette
plt.figure(figsize=(10, 6))
sns.barplot(x=profession_distribution.values, y=profession_distribution.index,
            hue=profession_distribution.index, palette='viridis', dodge=False)

# Add titles and labels
plt.title('Distribution of Professions Among Customers', fontsize=16)
plt.xlabel('Count', fontsize=14)
plt.ylabel('Profession', fontsize=14)

# Show the plot
plt.show()

```

✓ 1.0s

Figure 6

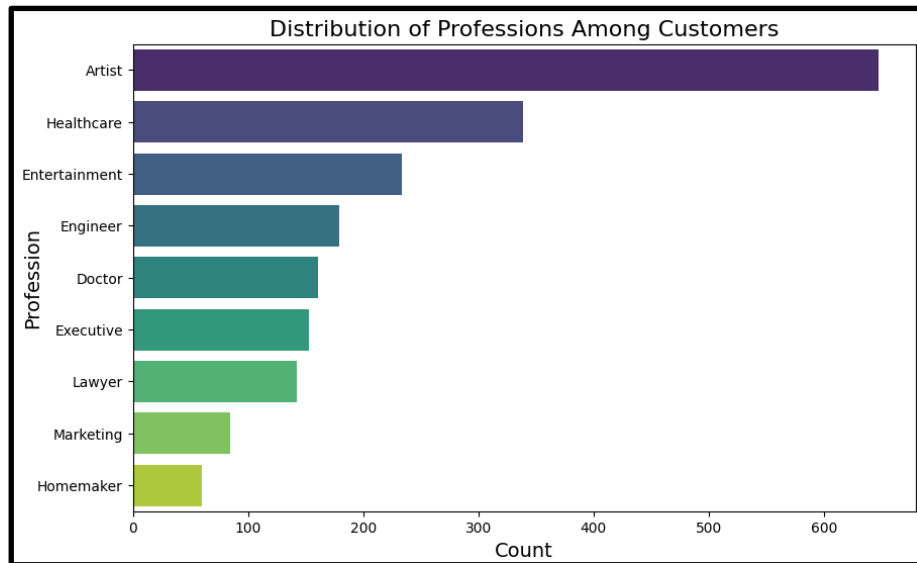


Figure 7

Outliers in 'Annual Income' and 'Spending Score' columns of the customer_data DataFrame were identified using Interquartile Range (IQR) method by calculating first (Q1) and third (Q3) quartiles, establishing lower and upper bounds for outlier detection, filtering for values outside these bounds, and keeping track of unique outlier indices (Figure 8).

```
# Define the features for analysis: Annual Income and Spending Score
features_name = ['Annual Income ($)', 'Spending Score (1-100)']
features = customer_data[features_name]

# List to store the indices of outliers
outliers_index = []

# Loop through each feature to identify outliers
for column in features_name:
    # Calculate the first (Q1) and third (Q3) quartiles
    Q1 = features[column].quantile(0.25)
    Q3 = features[column].quantile(0.75)

    # Calculate the Interquartile Range (IQR)
    IQR = Q3 - Q1

    # Determine the lower and upper bounds for outlier detection
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers based on the bounds
    outliers = features[(features[column] < lower_bound) | (features[column] > upper_bound)]

    # Collect the indices of identified outliers
    for idx in outliers.index:
        if idx not in outliers_index:
            outliers_index.append(idx)

    # Print the number of outliers found for the current feature
    print(f"Number of outliers in {column}: {len(outliers)}")
```

✓ 0.0s

Number of outliers in Annual Income (\$): 0
 Number of outliers in Spending Score (1-100): 0

Figure 8

For the clustering analysis, two feature sets were used: the first set comprised of 'Annual Income' and 'Spending Score', which depicts lifestyle, and the second set comprised of 'Age' and 'Family Size', which represents demographics. Both feature sets were normalised using MinMaxScaler to scale values between 0 and 1, ensuring that each variable contributed equally to the clustering process and no skewed results occurred due to differences in feature scale (Figure 9 and Figure 10).

```
# Create an instance of MinMaxScaler for normalization
minmax_scaler = MinMaxScaler()

# Fit and transform the features
normalized_features = minmax_scaler.fit_transform(features)

# Convert the normalized data back into a DataFrame with the same column names
normalized_features_df = pd.DataFrame(normalized_features, columns=features.columns)

# Calculate and display the peak-to-peak range for the raw numerical features
print(f"Peak to Peak range by column in Raw      :\n{np.ptp(features, axis=0)}")

# Calculate and display the peak-to-peak range for the normalized features
print(f"Peak to Peak range by column in Normalized :\n{np.ptp(normalized_features_df, axis=0)}")

# Print the normalized features
print(normalized_features_df)
```

✓ 0.0s

```
Peak to Peak range by column in Raw      :
Annual Income ($)      189974
Spending Score (1-100)    100
dtype: int64
Peak to Peak range by column in Normalized :
Annual Income ($)      1.0
Spending Score (1-100)    1.0
dtype: float64
   Annual Income ($)  Spending Score (1-100)
0          0.078958          0.39
1          0.184236          0.81
2          0.452694          0.06
3          0.310569          0.77
4          0.200027          0.40
...          ...          ...
1995         0.970591          0.40
1996         0.385095          0.32
1997         0.478808          0.14
1998         0.958600          0.04
1999         0.582238          0.52

[2000 rows x 2 columns]
```

Figure 9

```

# Select relevant features
customer_features = clustered_data[['Age', 'Family Size']]

# Create an instance of MinMaxScaler for normalization
minmax_scaler = MinMaxScaler()

# Fit and transform the features
normalized_customer_features = minmax_scaler.fit_transform(customer_features)

# Convert the normalized data back into a DataFrame with the same column names
normalized_features_df = pd.DataFrame(normalized_customer_features, columns=['Age', 'Family Size'])

# Calculate and display the peak-to-peak range for the raw numerical features
print(f"Peak to Peak range by column in Raw      :\n{np.ptp(customer_features, axis=0)}")

# Calculate and display the peak-to-peak range for the normalized features
print(f"Peak to Peak range by column in Normalized :\n{np.ptp(normalized_features_df, axis=0)}")

# Print the normalized features
print(normalized_features_df)

```

✓ 0.0s

```

Peak to Peak range by column in Raw      :
Age          99
Family Size    8
dtype: int64
Peak to Peak range by column in Normalized :
Age          1.0
Family Size  1.0
dtype: float64

```

	Age	Family Size
0	0.191919	0.375
1	0.212121	0.250
2	0.202020	0.000
3	0.232323	0.125
4	0.313131	0.625
...	...	...
1995	0.717172	0.750
1996	0.919192	0.750
1997	0.878788	0.125
1998	0.777778	0.125
1999	0.909091	0.125

[2000 rows x 2 columns]

Figure 10

I chose K-means clustering to segment customer data, with the optimal number of clusters (k) obtained using the Elbow Method, which consisted of calculating the within-cluster sum of squares (WCSS) for various k values to visually identify the “elbow” point (Figure 11), and Silhouette Analysis, which provided additional validation of the selected k (Figure 12). After defining optimal k values, the K-means algorithm was applied to segment customers, and descriptive labels were assigned to clusters such as ‘Budget-Conscious’ and ‘High Spending, Low Income’ based on Annual Income and Spending Score, and for

Age and Family Size clusters labels such as 'Young with Small Family' and 'Old with Large Family'.

```
def plot_elbow_method(data, max_clusters=10):
    """
    Plots the Elbow Method for determining the optimal number of clusters.

    Parameters:
    - data: Input data for clustering.
    - max_clusters: Maximum number of clusters to test (default is 10).
    """
    wcss = [] # List to store WCSS values

    for num_clusters in range(1, max_clusters + 1):
        kmeans = KMeans(n_clusters=num_clusters, init='k-means++', max_iter=300, n_init=10, random_state=0)
        kmeans.fit(data)
        wcss.append(kmeans.inertia_) # Append WCSS value

    plt.figure(figsize=(10, 6)) # Set figure size
    plt.plot(range(1, max_clusters + 1), wcss, marker='o') # Plot WCSS values
    plt.title('Elbow Method for Optimal Number of Clusters', fontsize=16)
    plt.xlabel('Number of Clusters', fontsize=14)
    plt.ylabel('WCSS', fontsize=14)
    plt.xticks(range(1, max_clusters + 1)) # Show x-ticks for clusters
    plt.grid(True) # Add grid for clarity
    plt.show() # Display the plot

# Call the function to plot the Elbow Method using the normalized features
plot_elbow_method(X_array)
```

✓ 0.7s

Figure 11

```
def calculate_and_plot_silhouette_scores(data, min_clusters=2, max_clusters=10):
    """
    Calculates silhouette scores for a range of cluster counts and plots the results.
    Setting min_clusters = 2 is needed for useful grouping, since silhouette scores don't work with one cluster.

    Parameters:
    - data: Input data for clustering.
    - min_clusters: Minimum number of clusters to evaluate (default is 2).
    - max_clusters: Maximum number of clusters to evaluate (default is 10).
    """
    silhouette_scores = [] # List to store silhouette scores

    for k in range(min_clusters, max_clusters + 1):
        kmeans = KMeans(n_clusters=k, init='k-means++', max_iter=300, n_init=10, random_state=0)
        kmeans.fit(data)
        score = silhouette_score(data, kmeans.labels_) # Calculate silhouette score
        silhouette_scores.append(score) # Append score to list

    # Plot the silhouette scores
    plt.figure(figsize=(10, 6))
    plt.plot(range(min_clusters, max_clusters + 1), silhouette_scores, marker='o')
    plt.xticks(range(min_clusters, max_clusters + 1)) # Set x-ticks for clusters
    plt.xlabel("Number of Clusters", fontsize=14)
    plt.ylabel("Silhouette Coefficient", fontsize=14)
    plt.title("Silhouette Coefficients for Different Numbers of Clusters", fontsize=16)
    plt.grid(True) # Add grid for better readability
    plt.show() # Display the plot

# Call the function to calculate and plot silhouette scores
calculate_and_plot_silhouette_scores(X_array)
```

✓ 1.4s

Figure 12

The code shown in Figure 13 and Figure 14 implements K-means algorithm on customer data, focusing on clustering customers based on lifestyle factors such as Annual Income and Spending Score and demographic factors such as Age and Family Size. It sets parameters for clustering, assesses which customers fall into which clusters, and presents output in the form of scatter plots where each cluster is represented using a different color while centroids are highlighted in order to improve the understanding of the characteristics and the grouping of the customers.

```
# Set parameters for KMeans clustering
num_clusters = 7
max_iterations = 500
num_init = 10

# Perform KMeans clustering on the data
kmeans = KMeans(n_clusters=num_clusters, max_iter=max_iterations, n_init=num_init, random_state=0)
kmeans_predictions = kmeans.fit_predict(X_array)

# Display the KMeans cluster predictions
display(kmeans_predictions)

✓ 0.1s
array([0, 5, 3, ..., 3, 6, 4])

# Colors for the 7 clusters
colors = ['cyan', 'blue', 'black', 'green', 'yellow', 'red', 'purple']

# Cluster labels
labels = [
    'High Spending, Low Income',
    'Budget-Conscious',
    'Affluent High Spenders',
    'Low Spending, Low Income',
    'Moderate Spenders',
    'Luxury Spenders with Limited Income',
    'Cautious Spenders'
]

# Scatter plot of client lifestyle clusters based on Annual Income and Spending Score, with centroids highlighted.
plt.figure(figsize=(12, 9))
for i in range(7):
    plt.scatter(X_array[kmeans_predictions == i, 0], X_array[kmeans_predictions == i, 1], c=colors[i], label=labels[i], s=20)

plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=200, c='orange', label='Centroids')
plt.title('Client Lifestyle Clusters')
plt.xlabel('Annual Income ($)')
plt.ylabel('Spending Score (1-100)')
plt.legend(loc='best')
plt.show()

✓ 0.8s
```

Figure 13

```

# Choose 4 as optimal number of clusters
optimal_k = 4
kmeans = KMeans(n_clusters=optimal_k, random_state=0)
clustered_data['Cluster'] = kmeans.fit_predict(normalized_features_df)

# Map numerical clusters to descriptive names
cluster_names = {
    0: 'Old, with Large Family',
    1: 'Young, with Small Family',
    2: 'Old, with Small Family',
    3: 'Young, with Large Family'
}
clustered_data['Demographics'] = clustered_data['Cluster'].replace(cluster_names)

# Plotting the clusters with cluster names
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Age', y='Family Size', hue='Demographics', data=clustered_data, palette='Set2', alpha=0.9)

# Calculate and plot the centroids
centers = minmax_scaler.inverse_transform(kmeans.cluster_centers_)
centers_df = pd.DataFrame(centers, columns=['Age', 'Family Size'])
plt.scatter(centers_df['Age'], centers_df['Family Size'], c='red', marker='X', s=200, label='Centroids')

# Adding titles and labels
plt.title('Clustering Customers by Age and Family Size')
plt.xlabel('Age')
plt.ylabel('Family Size')
plt.legend(title='Demographics')
plt.show()

```

✓ 0.5s

Figure 14

This structured methodology I used combines rigorous data preprocessing with K-means clustering to achieve precise customer segmentation. Employing the Elbow Method to determine the number of clusters available helps in this methodology to create actionable insights, enabling strategies for engaging with customers in a more effective manner. According to a study by Baiq Nikum Yuliasih et al. (2024), a structured methodology was employed to achieve accurate customer segmentation through data preprocessing and K-means clustering. The research aims to develop a K-means clustering model to classify customers according to their potential for purchasing more products in an effort to keep the current customers and attract new ones, with a novel application of K-means to improve the customer profile based on purchasing behaviour (Baiq Nikum Yuliasih, et al., 2024).

Results and Discussion

Through data preprocessing, exploratory analysis, feature engineering, clustering, and visualisation outlined in the Methodology, I derived insightful customer segments, which are detailed in the findings. Figure 15 depicts the dataset before any processing.

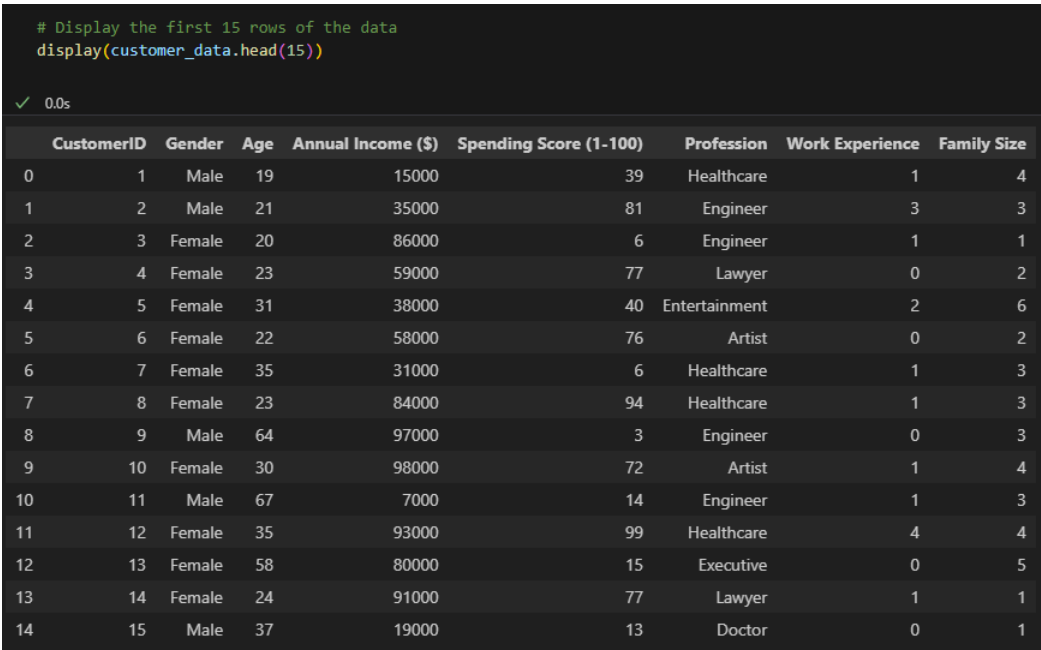


Figure 15

Through initial exploration and data inspection as demonstrated in the Methodology, it was revealed that the ‘Profession’ column contained some missing values since there are a total of 2000 customers (Figure 16). The missing values were then filled in with the mode in order to achieve data completeness (Figure 17).



Figure 16

	CustomerID	Gender	Age	Annual Income (\$)	Spending Score (1-100)	Profession	Work Experience	Family Size	Original_Profession	Filled_Profession
79	80	Female	49	98000	42	Artist	1	1	NaN	Artist
118	119	Female	51	84000	43	Artist	2	7	NaN	Artist
219	220	Female	59	76000	61	Artist	9	1	NaN	Artist
237	238	Male	95	36000	35	Artist	0	4	NaN	Artist
437	438	Male	76	136259	14	Artist	0	7	NaN	Artist
440	441	Female	0	57373	29	Artist	0	7	NaN	Artist
498	499	Male	95	121725	3	Artist	12	3	NaN	Artist
545	546	Female	89	107359	26	Artist	10	6	NaN	Artist
601	602	Male	61	126370	20	Artist	11	4	NaN	Artist
641	642	Male	66	121377	19	Artist	7	7	NaN	Artist
665	666	Male	28	101414	64	Artist	8	1	NaN	Artist
703	704	Male	22	114011	40	Artist	5	7	NaN	Artist
801	802	Male	81	148208	36	Artist	5	7	NaN	Artist
817	818	Female	91	154456	71	Artist	0	7	NaN	Artist
850	851	Male	69	186655	32	Artist	7	2	NaN	Artist
903	904	Female	15	174501	37	Artist	9	7	NaN	Artist
927	928	Male	25	81367	87	Artist	0	3	NaN	Artist
1009	1010	Male	69	61637	67	Artist	0	5	NaN	Artist
1067	1068	Female	30	78821	46	Artist	1	4	NaN	Artist
1088	1089	Female	10	162076	78	Artist	0	3	NaN	Artist
1092	1093	Male	87	141383	3	Artist	7	5	NaN	Artist
1188	1189	Female	99	122548	14	Artist	2	5	NaN	Artist
1223	1224	Male	74	184838	18	Artist	8	5	NaN	Artist
1316	1317	Female	33	109396	17	Artist	1	2	NaN	Artist
1334	1335	Male	42	94844	64	Artist	7	6	NaN	Artist
1384	1385	Female	50	104467	72	Artist	5	1	NaN	Artist
1417	1418	Female	1	138656	27	Artist	3	4	NaN	Artist
1483	1484	Female	19	147052	86	Artist	9	4	NaN	Artist
1506	1507	Female	9	173233	18	Artist	5	1	NaN	Artist
1625	1626	Female	78	130706	72	Artist	3	6	NaN	Artist
1634	1635	Male	73	97455	55	Artist	6	4	NaN	Artist
1788	1789	Female	58	51416	46	Artist	7	5	NaN	Artist
1884	1885	Female	63	155638	81	Artist	1	4	NaN	Artist
1905	1906	Female	52	164038	53	Artist	5	4	NaN	Artist
1932	1933	Female	23	59905	79	Artist	7	5	NaN	Artist
	CustomerID	Gender	Age	Annual Income (\$)	Spending Score (1-100)	Profession	Work Experience	Family Size		

Figure 17

Figure 18 depicts summary statistics for customer data.

```
# Summary statistics for numerical variables in the customer data
customer_data.describe()
```

✓ 0.0s

	CustomerID	Age	Annual Income (\$)	Spending Score (1-100)	Work Experience	Family Size
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	1000.500000	48.960000	110731.821500	50.962500	4.102500	3.768500
std	577.494589	28.429747	45739.536688	27.934661	3.922204	1.970749
min	1.000000	0.000000	0.000000	0.000000	0.000000	1.000000
25%	500.750000	25.000000	74572.000000	28.000000	1.000000	2.000000
50%	1000.500000	48.000000	110045.000000	50.000000	3.000000	4.000000
75%	1500.250000	73.000000	149092.750000	75.000000	7.000000	5.000000
max	2000.000000	99.000000	189974.000000	100.000000	17.000000	9.000000

Figure 18

Then the data were visualised in histograms and bar plots (Figure 19 and Figure 20).

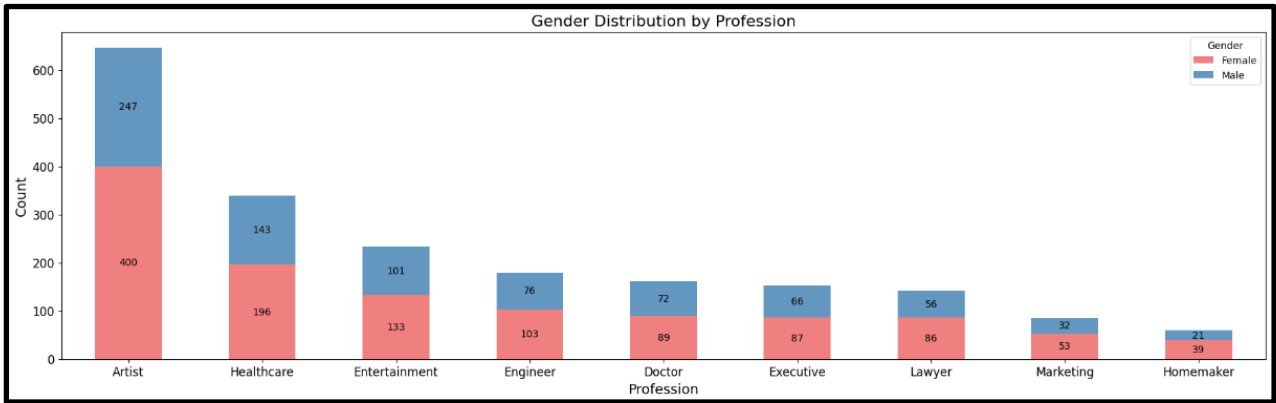


Figure 19

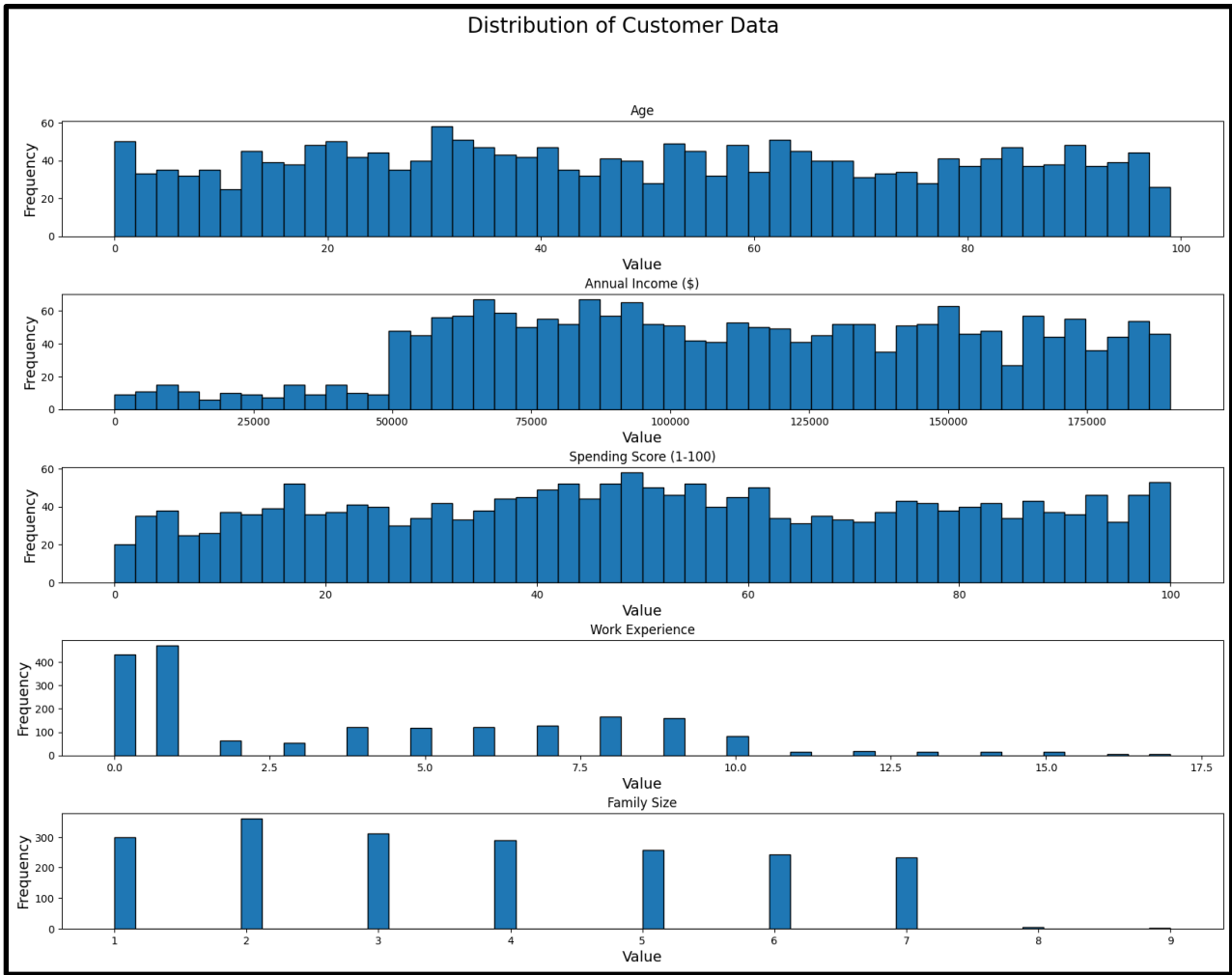


Figure 20

All feature values were brought to a consistent range using MinMax scaling for normalisation. This enhanced the clustering process by ensuring that no single feature disproportionately influenced the distance calculations central to K-Means clustering. Figures 21 and 22 depict statistics for the normalised features. The data that was normalised revealed a more cohesive distribution, improving the interpretability of the cluster results (Figure 23 and Figure 24).

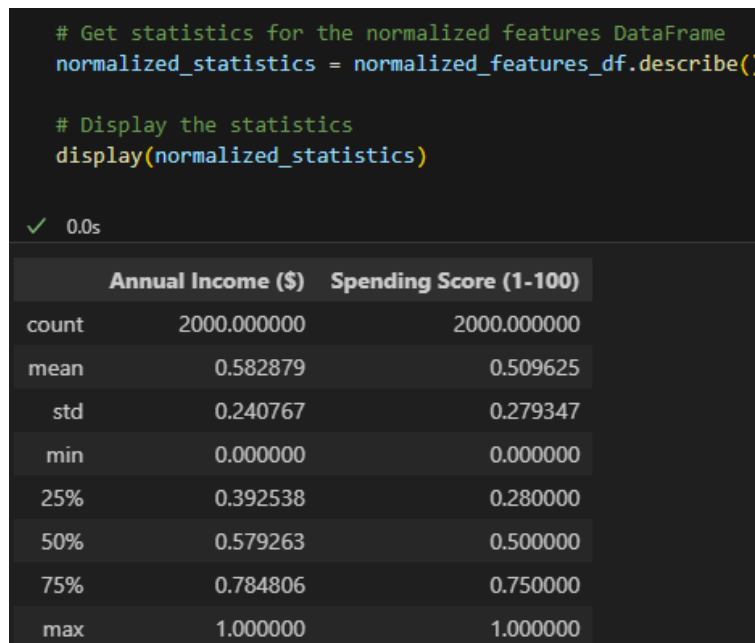


Figure 21

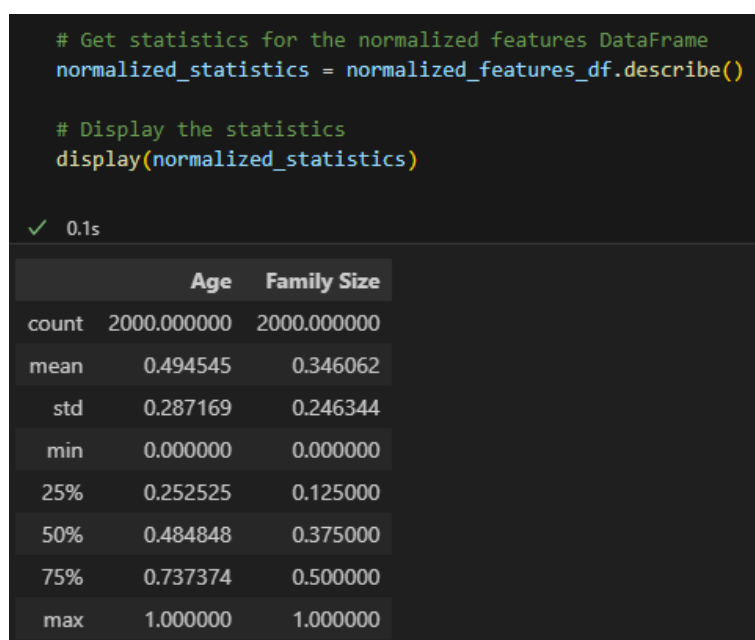


Figure 22

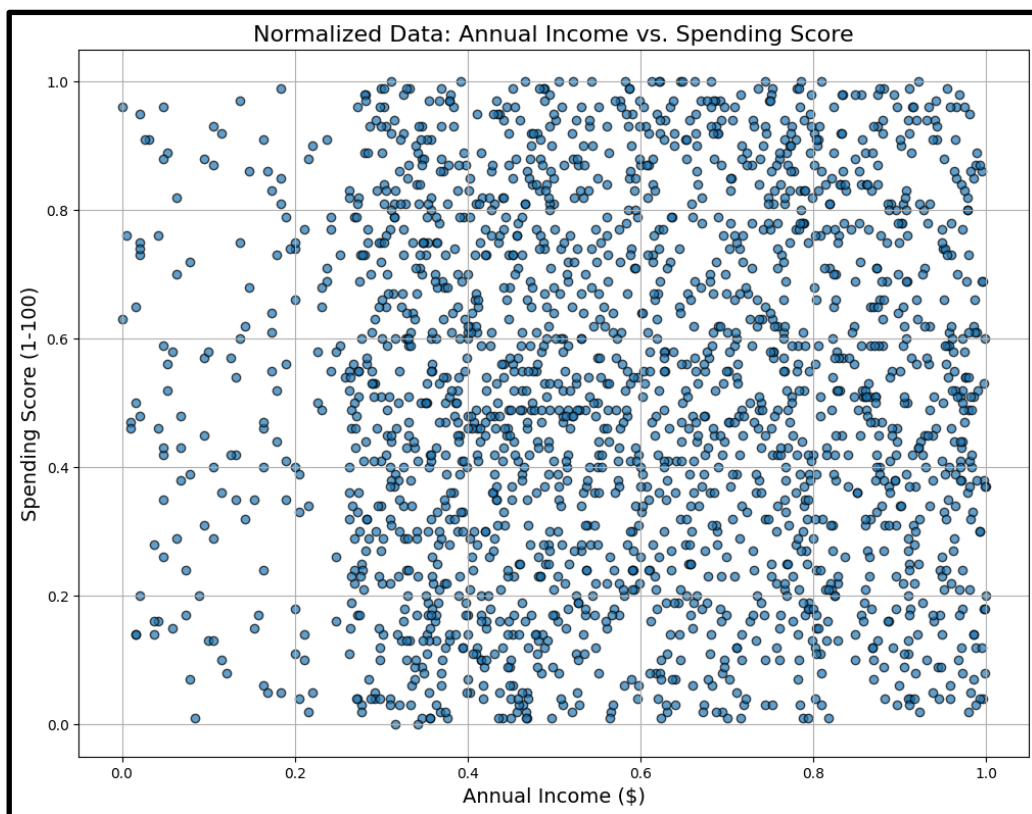


Figure 23

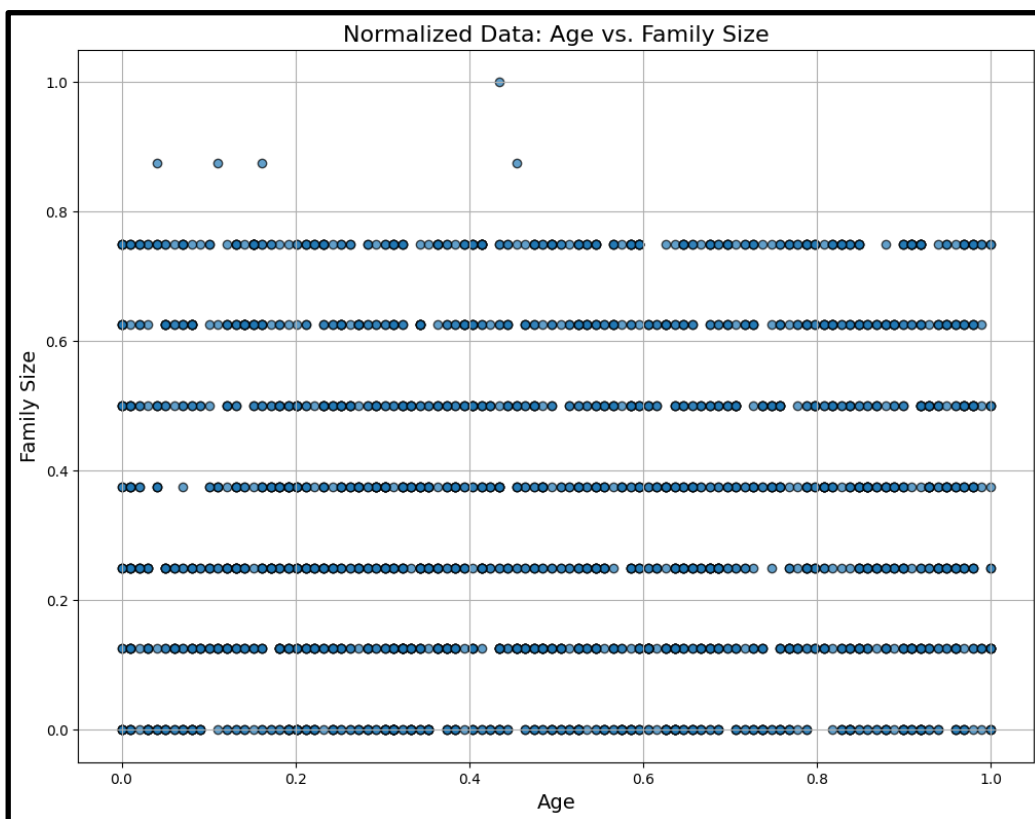


Figure 24

The optimal number of clusters was determined using the Elbow Method and Silhouette Analysis. The Elbow Method suggested that for lifestyle segmentation (based on Annual Income and Spending Score), the most appropriate number of clusters would be 4 (Figure 25), while for demographic segmentation (Age and Family size) 4 clusters would be appropriate (Figure 26), as determined by plotting the number of clusters against the within-cluster sum of squares (WCSS).

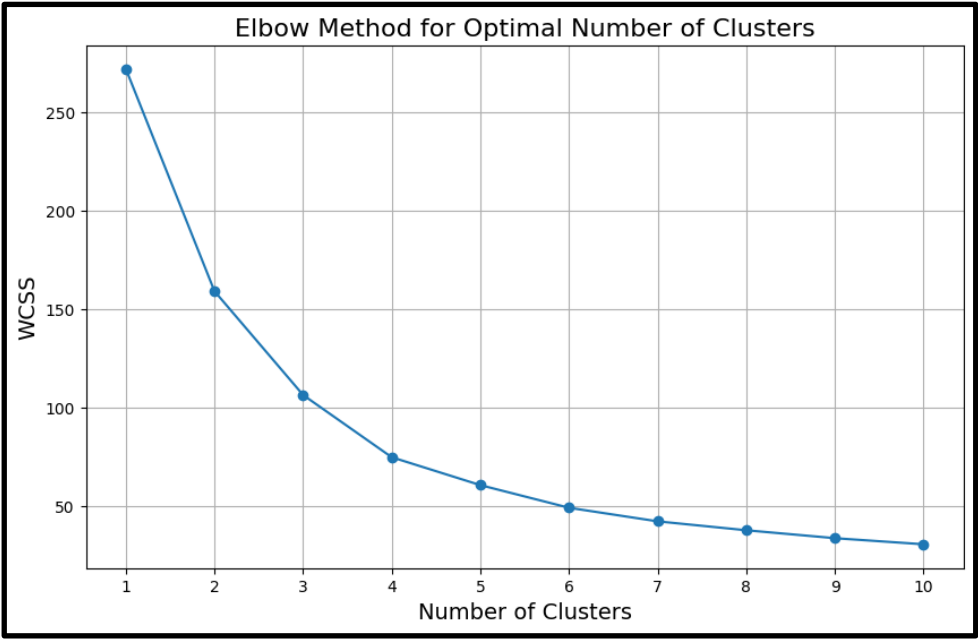


Figure 25

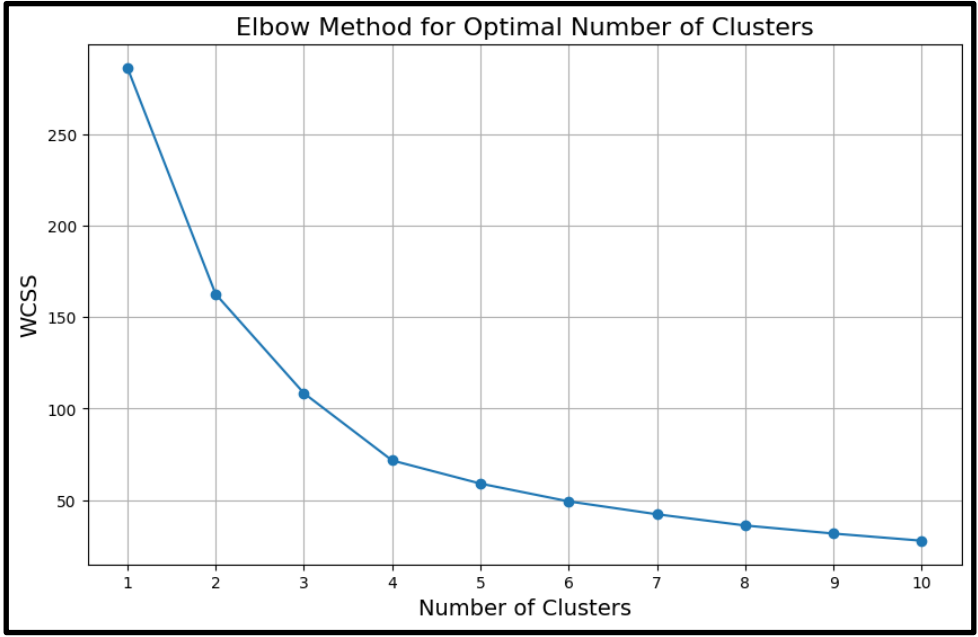


Figure 26

In order to confirm the validity of the clusters, silhouette scores were calculated for each cluster count. It was observed from the results that a strong clustering structure was achieved with 7 and 4 numbers of clusters, respectively (Figure 27 and Figure 28).

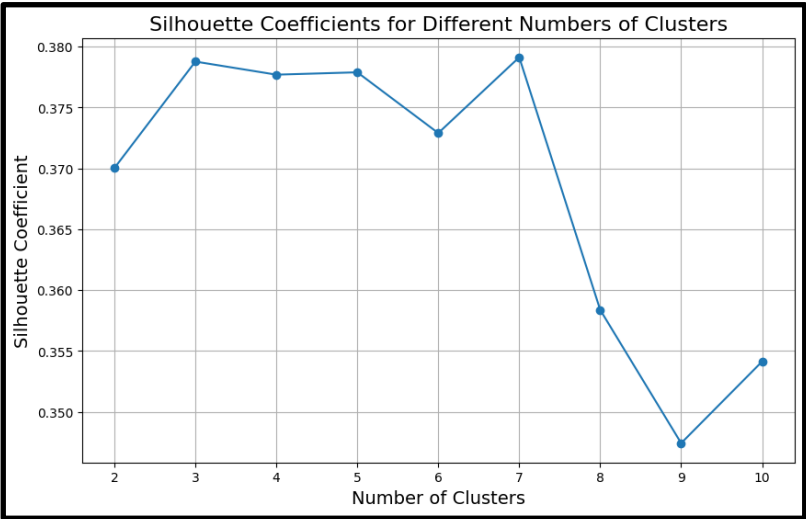


Figure 27

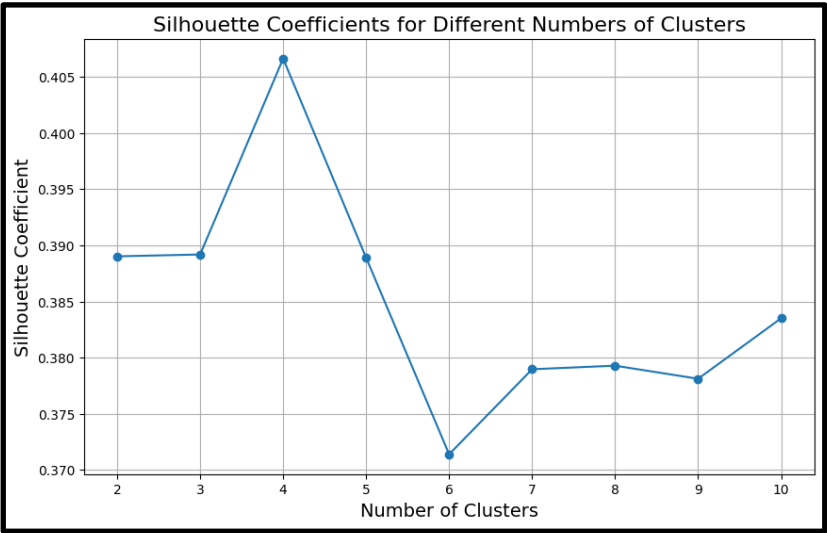


Figure 28

In the case of lifestyle segmentation, although the Elbow Method suggested 4 clusters as an optimal point, the Silhouette Score indicated that 7 clusters provide a stronger clustering structure. I chose 7 clusters to maximise the Silhouette Score, so that this number of clusters can result in more cohesive and well-separated clusters compared to 4 clusters. After determining the optimal clusters, the data was then divided according

to lifestyle and demographic characteristics, resulting in groupings that are significant and meaningful when it comes to targeting the customers (Figure 29 and Figure 30).

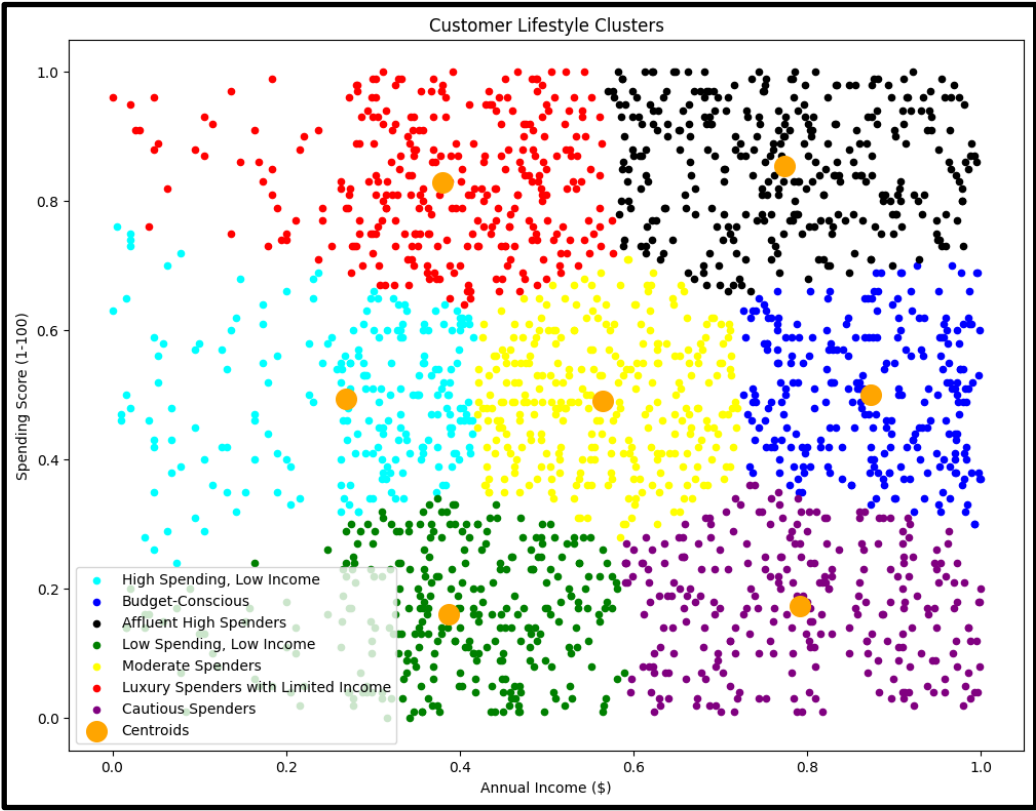


Figure 29

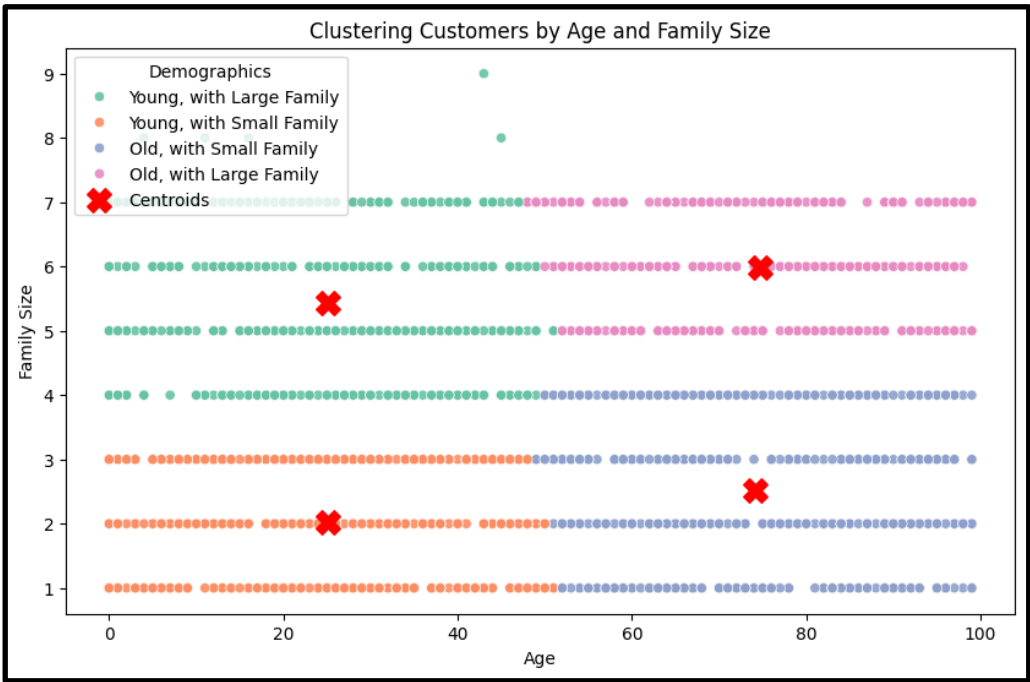


Figure 30

Every cluster serves as an illustration of different behaviours or preferences of customers, which can help to design appropriate marketing strategies, suggest products to customers, and even allocate resources more effectively. Additionally, demographic segmentation complements the lifestyle segmentation, providing more useful information based on the age and family size of the customers. In Figure 31, the updated customer data adds descriptive names on various clusters, which makes it easier to comprehend and analyse various types of customers, helping in targeted marketing strategies.

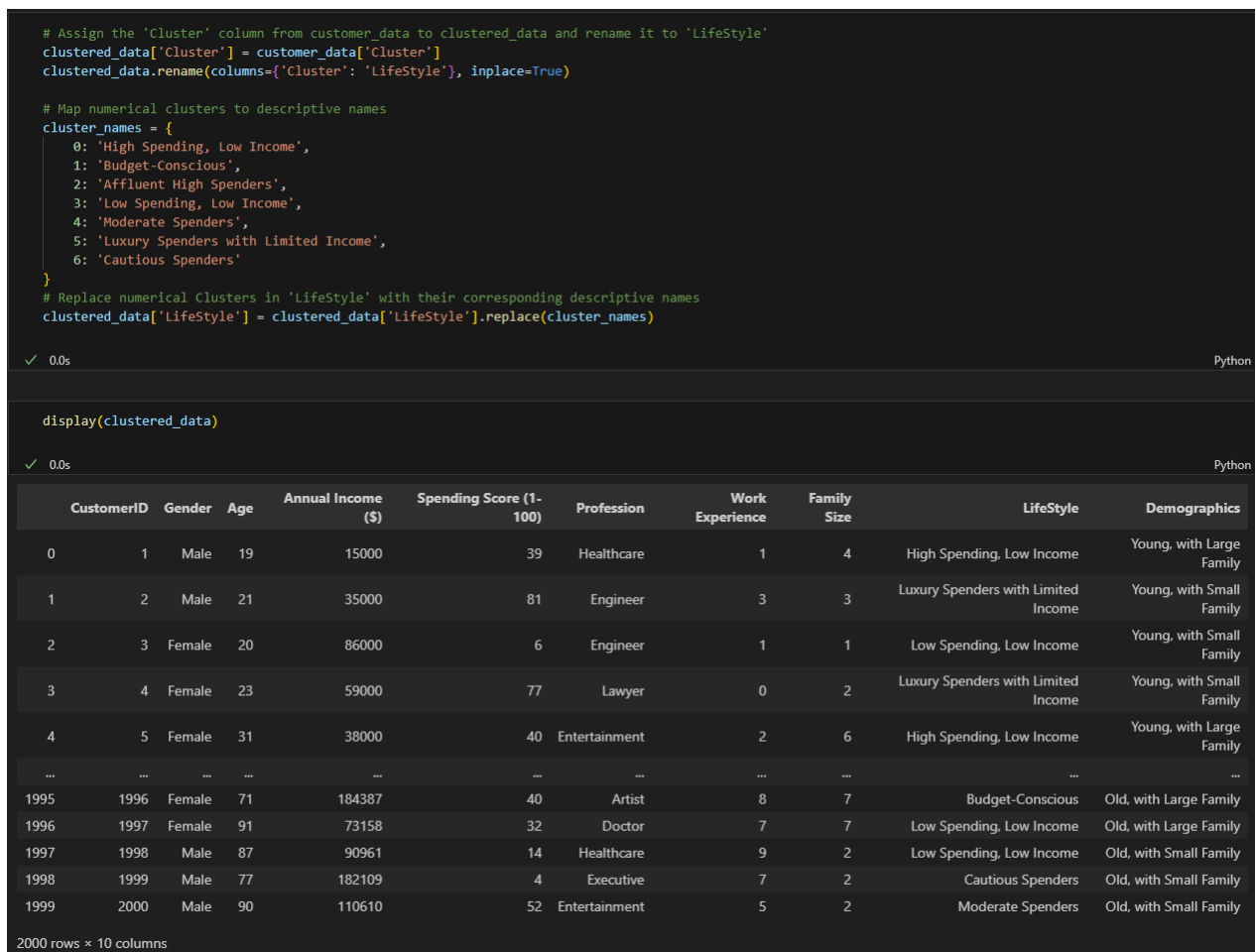


Figure 31

For example, filtering for 'Cautious Spenders' and 'Old, with Small Family' helps identify a specific customer segment (Figure 32). Figure 33 and Figure 34 depict the visualised data in histograms and bar plots.

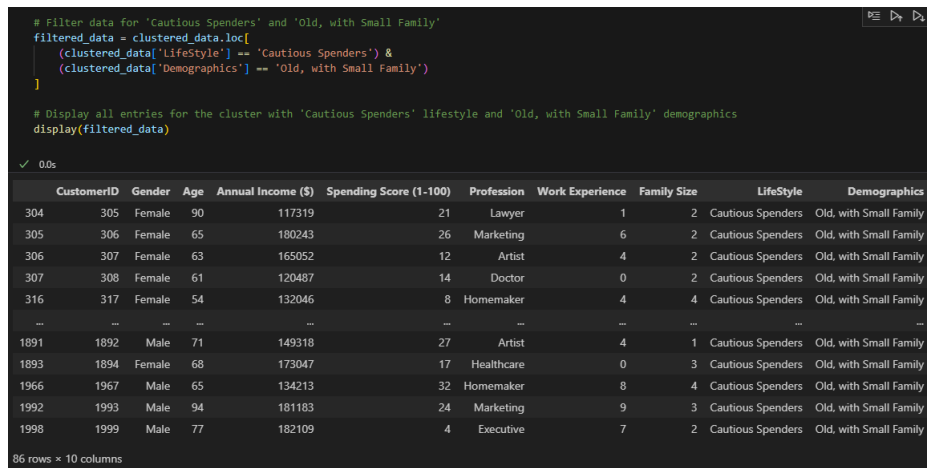


Figure 32

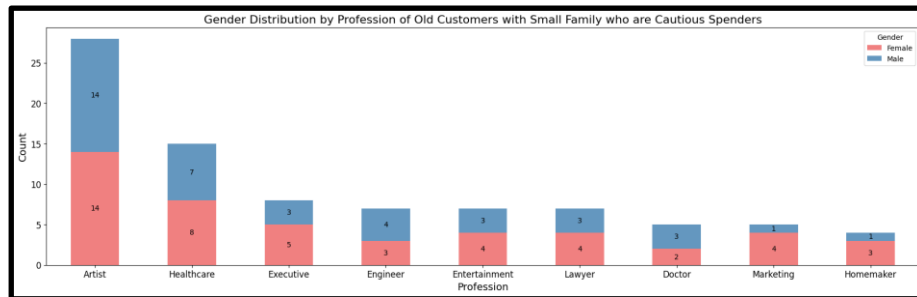


Figure 33

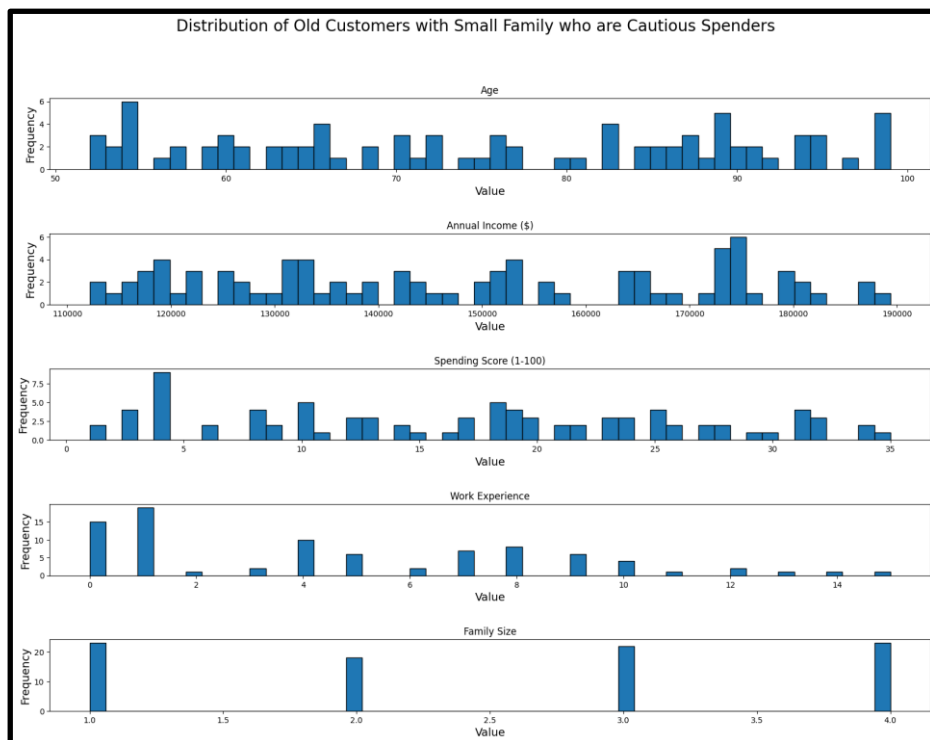


Figure 34

The findings from this project indicate that K-Means clustering is useful for understanding customer behaviour, enabling businesses and companies to accurately target their audience. Strategies that target each niche can be devised, increasing customer engagement and overall satisfaction. For example, the 'High Spending, Low Income' cluster could be engaged with flexible payment plans, while luxury product advertising would work well for the targeted cluster of 'Affluent High Spenders'. In addition, demographic segmentation can also help customise product recommendations, improve customer loyalty, and optimise marketing efforts. In terms of business, this is a good strategy as it provides more detailed information which can improve the positioning of the products, inventory management, and pricing strategies. It can be assumed that customers belonging to low spending clusters would be more responsive to discounts and promotions as opposed to customers in high spending clusters who would prefer loyalty rewards and high-end products. Overall, customer segmentation is essential for competitive advantage enabling businesses and companies to leverage the market with a more focused strategy.

This project demonstrates the effectiveness of K-Means clustering for customer segmentation but at the same time highlights limitations that could impact its accuracy and scope. K-means' tendency to form spherical clusters may overlook complex data relationships. Future improvements could involve experimenting with alternative clustering methods, such as DBSCAN or hierarchical clustering, which could capture varied data shapes and enhance the stability of the model. Additionally, trying different scaling methods could refine the model's robustness. These changes would facilitate the development of more personalised marketing plans, thereby improving customer interaction and loyalty.

References:

Books and documents;

- Akar, E., (2021). Customers' Online Purchase Intentions and Customer Segmentation During the Period of COVID-19 Pandemic. *Journal of Internet Commerce*, 20(3), pp. 371-401.
- Alves Gomes, M. & Meisen, T., (2023). A review on customer segmentation methods for personalized customer targeting in e-commerce use cases. *Information Systems and e-Business Management*, 21(3), pp. 527-570.
- Baiq Nikum Yulisasih, Herman, H. & Sunardi, S., (2024). K-Means Clustering Method For Customer Segmentation Based On Potential Purchases. *Jurnal ELTIKOM*, 8(1), pp. 83-90. Available at: <https://eltikom.poliban.ac.id/index.php/eltikom/article/view/1137/134>
- Madhu, J., Revanakar, K. K., Lavanya & Akash, (2021). Customer Segmentation using K-means Clustering. *International Journal of Advances in Engineering and Management (IJAEM)*, 3(7), pp. 2381-2386.
- Tabianan, K., Velu, S. & Ravi, V., (2022). K-Means Clustering Approach for Intelligent Customer Segmentation Using Customer Purchase Behavior Data. *Sustainability*, 14(12), p. 7243. Available at: <https://www.mdpi.com/2071-1050/14/12/7243>
- Zhao, J., (2024). Customer segmentation application based on K-Means. *Applied and Computational Engineering*, 47(1), pp. 242-247.